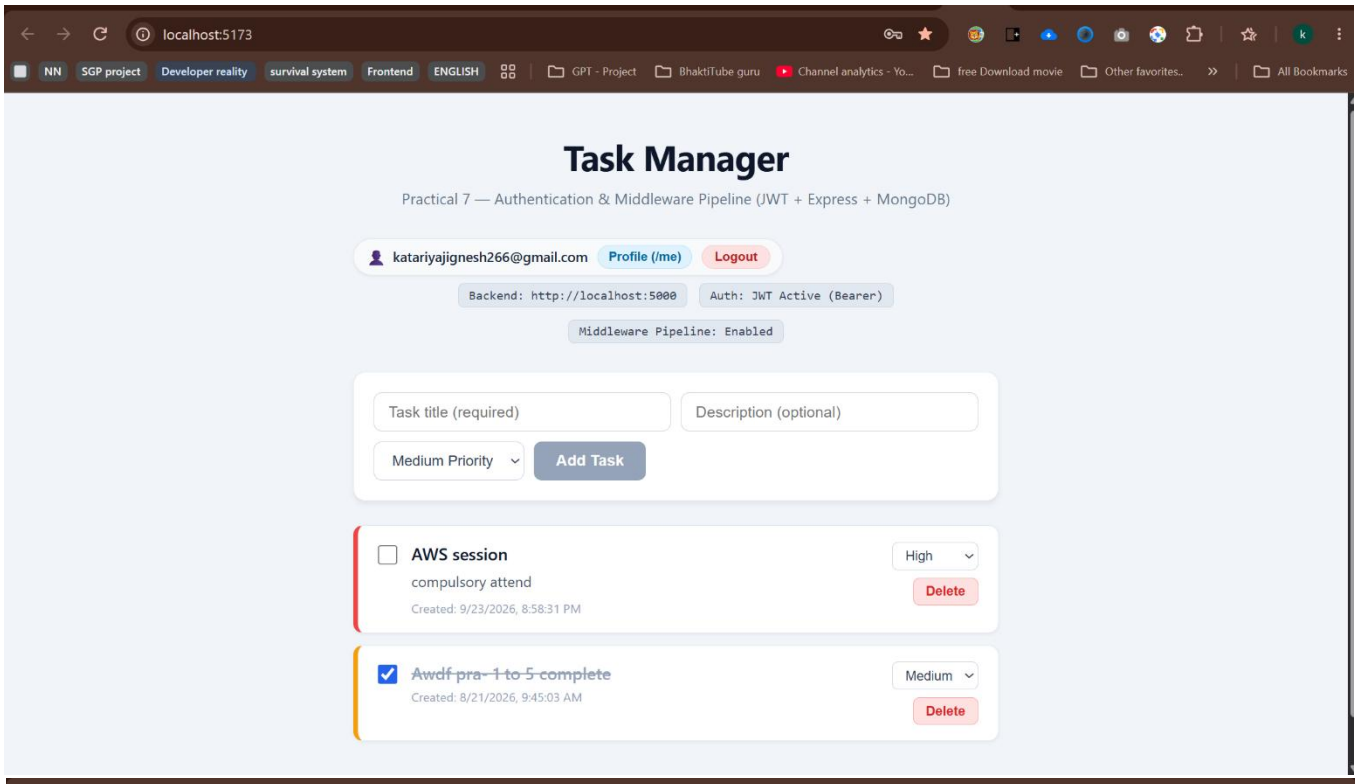
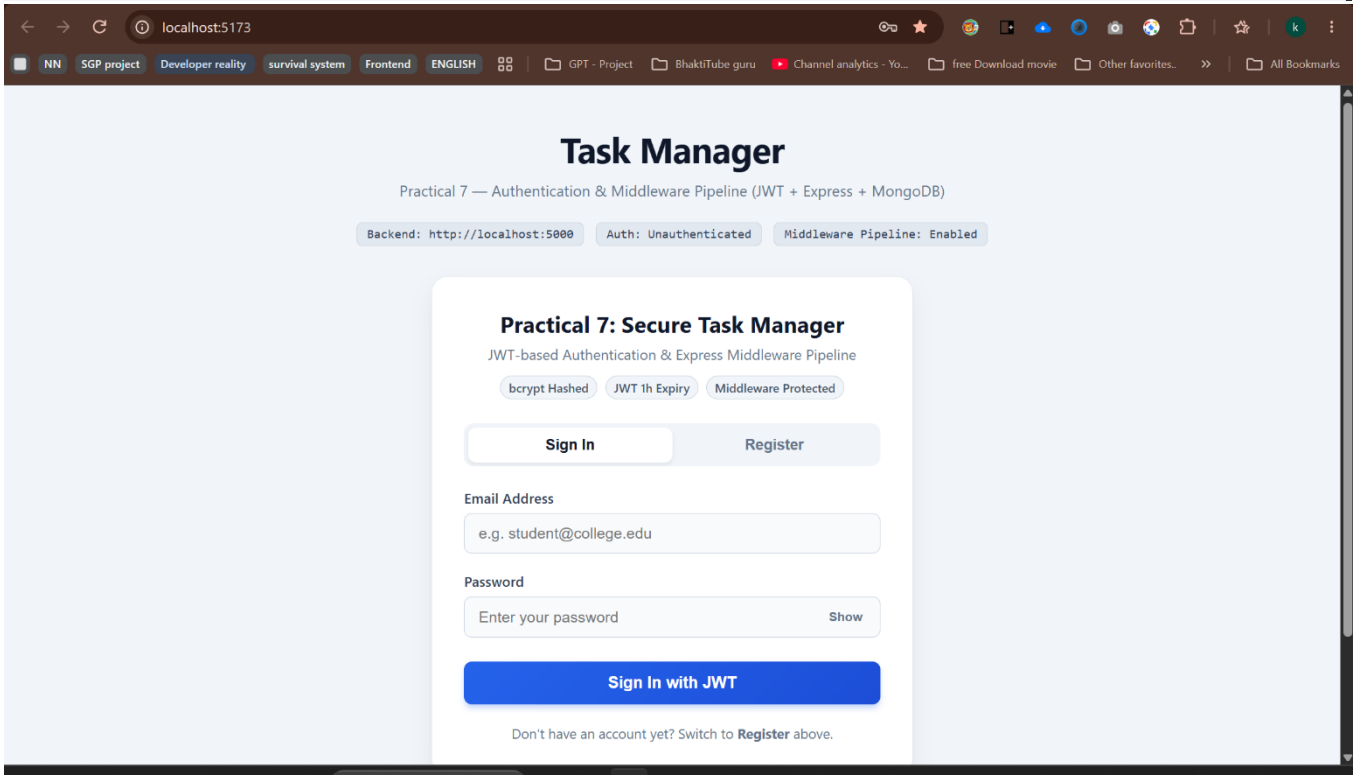
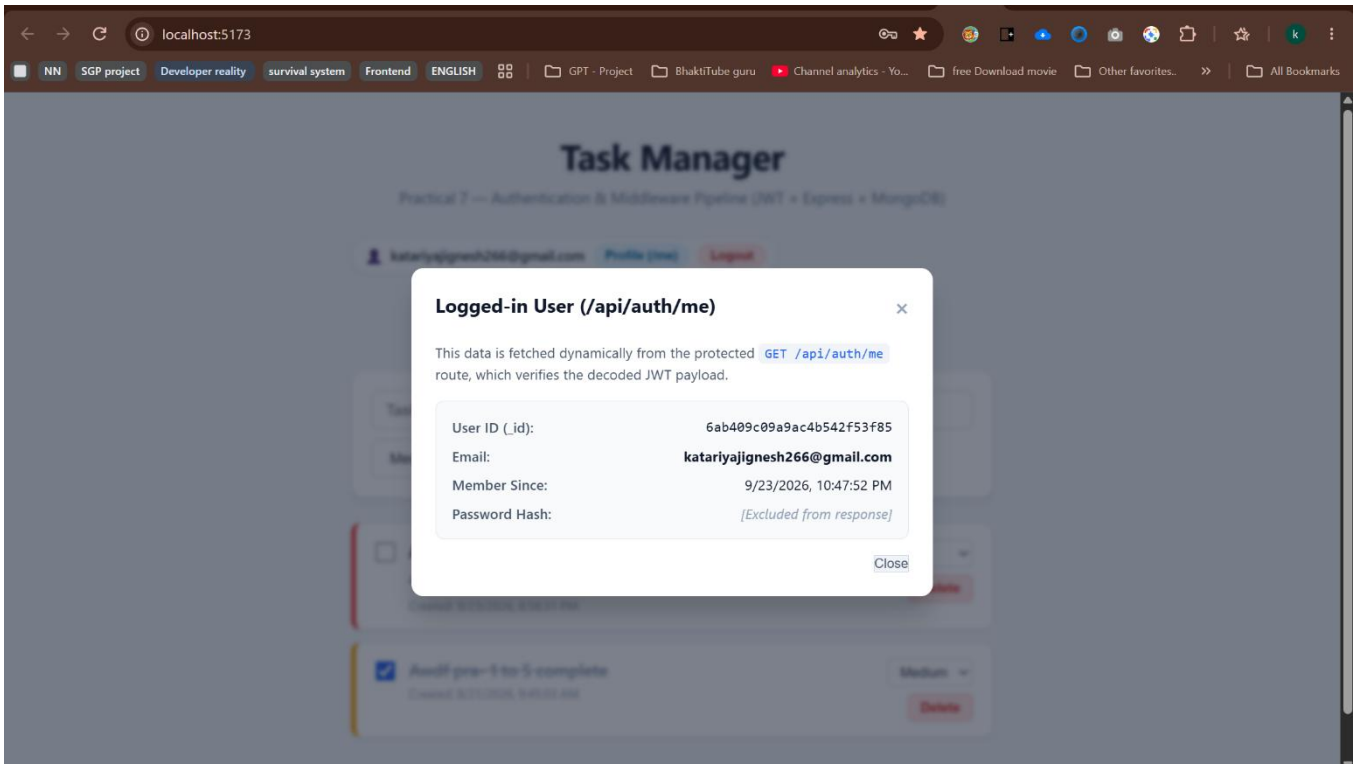
Practical 7: Authentication and Middleware Pipeline

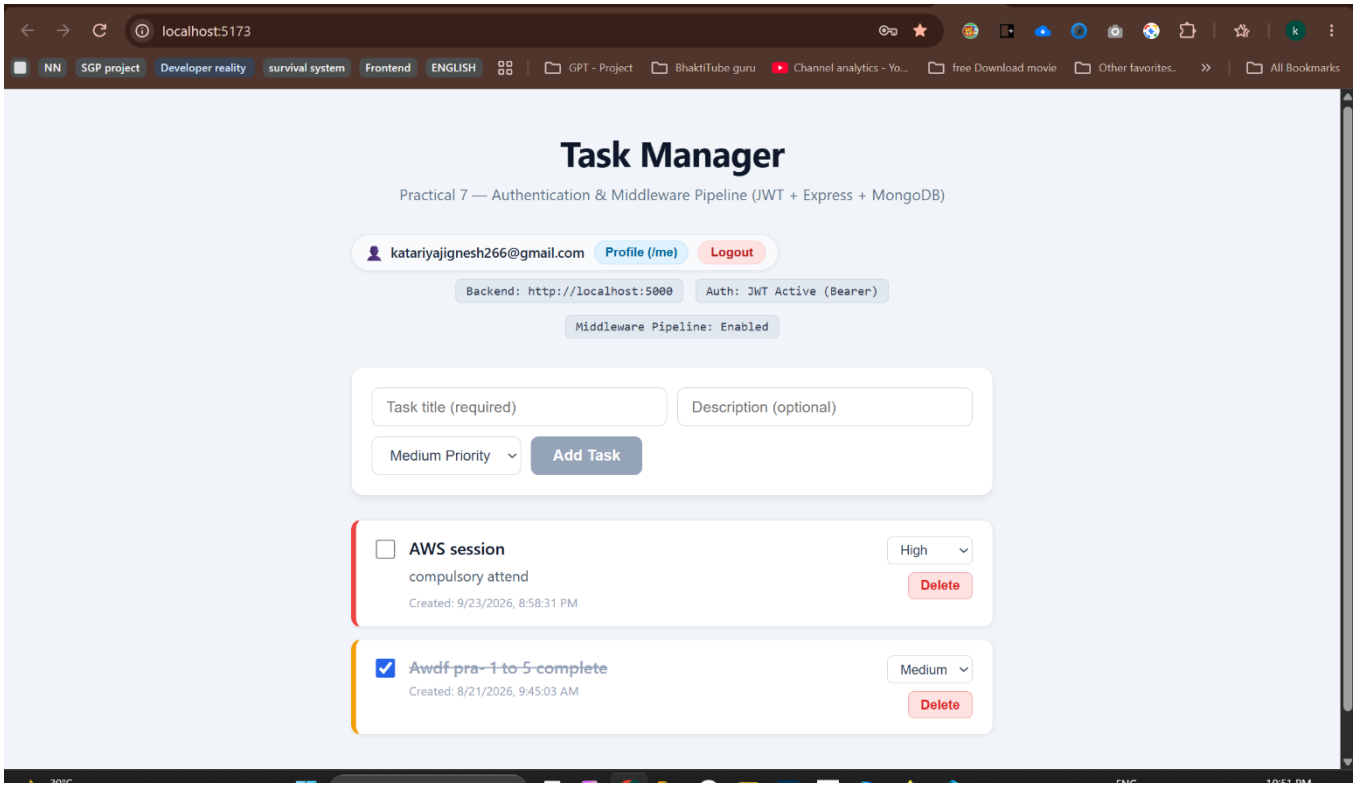
Objective	To implement JWT-based authentication and input validation as part of the Express middleware pipeline.
Problem Statement	• Add user registration and login functionality to the Task Management application. • Hash passwords using bcrypt before saving the User document to MongoDB. • Implement JWT token generation on successful login, with a reasonable expiry (e.g., 1 hour). • Protect all task routes using an authentication middleware that verifies the JWT. • Add server-side input validation that rejects malformed requests (e.g., missing title) before they reach the database.
Diagram	POST /register → hash password → save User → 201 POST /login → verify password → sign JWT → return token Protected Route Flow: Client Request (Authorization: Bearer <token>) | v [Auth Middleware] → verifies token → req.user set | v [Validation Middleware] → checks input | v Controller (task routes)

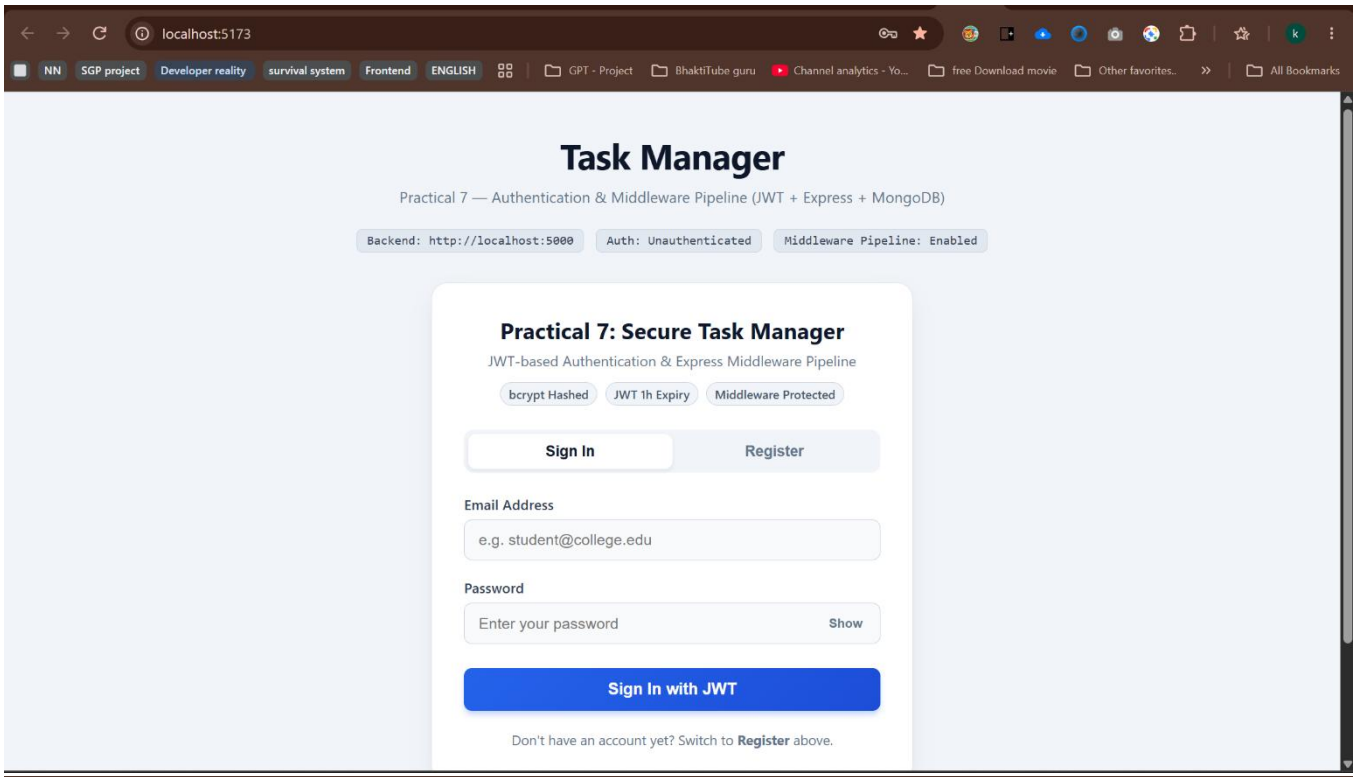
	1. Install required packages:
	npm install bcryptjs jsonwebtoken
	2. Create a User schema with email and hashed password fields.
	3. Implement the register route hash the password before saving:
	const hashedPassword = await bcrypt.hash(password, 10);
	const user = await User.create({ email, password: hashedPassword
	});
	4. Implement the login route compare password and sign a JWT:
	const isMatch = await bcrypt.compare(password, user.password);
	const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET,
	{ expiresIn: '1h' });
	5. Create an auth middleware that verifies the token from the Authorization header:
	const token = req.headers.authorization?.split(' ')[1];
	const decoded = jwt.verify(token, process.env.JWT_SECRET);
	req.user = decoded;
	6. Apply the auth middleware to all task routes.
	7. Add a validation middleware that checks required fields exist before reaching the controller.
	8. Test the full flow in Postman: register → login → copy token → access protected task routes.

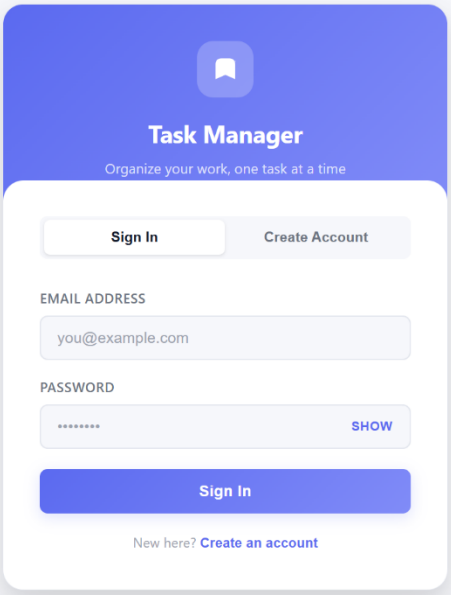

localhost:5173
Task Manager
Practical 7 — Authentication & Middleware Pipeline (JWT + Express + MongoDB)
katariyajignesh266@gmail.com
Profile (/me)
Logout
Backend: http://localhost:5000
Auth: JWT Active (Bearer)
Middleware Pipeline: Enabled
Task title (required)
Description (optional)
Medium Priority
Add Task
AWS session
compulsory attend
Created: 9/23/2026, 8:58:31 PM
High
Delete
Awdf pra- 1 to 5 complete
Created: 8/21/2026, 9:45:03 AM
Medium
Delete

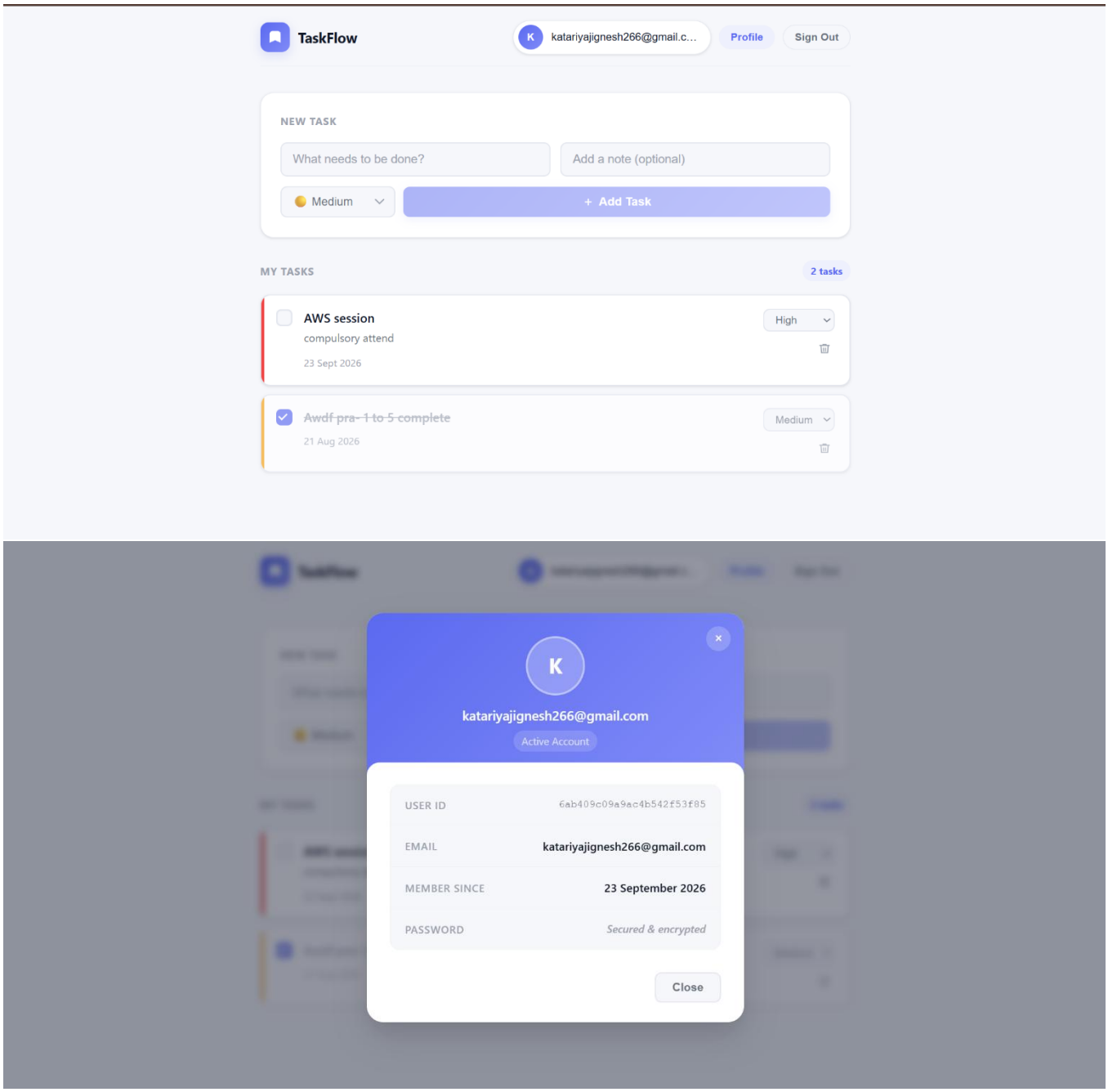

localhost:5173
Task Manager
Practical 7 — Authentication & Middleware Pipeline (JWT + Express + MongoDB)
Backend: http://localhost:5000
Auth: Unauthenticated
Middleware Pipeline: Enabled
Practical 7: Secure Task Manager
JWT-based Authentication & Express Middleware Pipeline
bcrypt Hashed
JWT 1h Expiry
Middleware Protected
Sign In
Register
Email Address
e.g. student@college.edu
Password
Enter your password
Show
Sign In with JWT
Don't have an account yet? Switch to Register above.

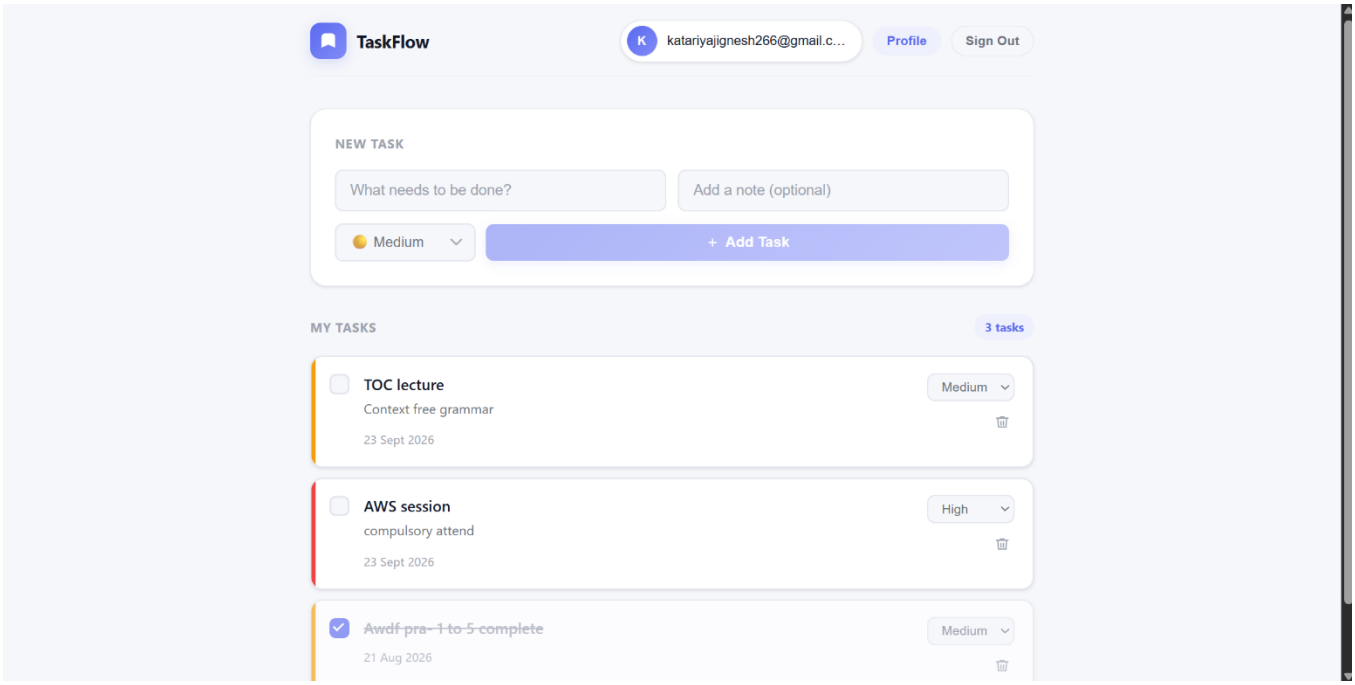

localhost:5173
Task Manager
Logged-in User (/api/auth/me)
This data is fetched dynamically from the protected GET /api/auth/me route, which verifies the decoded JWT payload.
User ID (_id): 6ab409c09a9ac4b542f53f85
Email: katariyajignesh266@gmail.com
Member Since: 9/23/2026, 10:47:52 PM
Password Hash: [Excluded from response]
Close


localhost:5173
Task Manager
Practical 7 — Authentication & Middleware Pipeline (JWT + Express + MongoDB)
katariyajignesh266@gmail.com
Profile (/me)
Logout
Backend: http://localhost:5000
Auth: JWT Active (Bearer)
Middleware Pipeline: Enabled
Task title (required)
Description (optional)
Medium Priority
Add Task
AWS session
compulsory attend
Created: 9/23/2026, 8:58:31 PM
High
Delete
Awdf pra-1 to 5 complete
Created: 8/21/2026, 9:45:03 AM
Medium
Delete


localhost:5173
NN
SGP project
Developer reality
survival system
Frontend
ENGLISH
GPT - Project
BhaktiTube guru
Channel analytics - Yo...
free Download movie
Other favorites..
All Bookmarks
Task Manager
Practical 7 — Authentication & Middleware Pipeline (JWT + Express + MongoDB)
Backend: http://localhost:5000
Auth: Unauthenticated
Middleware Pipeline: Enabled
Practical 7: Secure Task Manager
JWT-based Authentication & Express Middleware Pipeline
bcrypt Hashed
JWT 1h Expiry
Middleware Protected
Sign In
Register
Email Address
e.g. student@college.edu
Password
Enter your password
Show
Sign In with JWT
Don't have an account yet? Switch to Register above.


Task Manager
Organize your work, one task at a time
Sign In
Create Account
EMAIL ADDRESS
you@example.com
PASSWORD
SHOW
Sign In
New here? Create an account


TaskFlow
katariyajignesh266@gmail.c...
Profile
Sign Out
NEW TASK
What needs to be done?
Add a note (optional)
Medium
+ Add Task
MY TASKS
2 tasks
AWS session
compulsory attend
23 Sept 2026
High
Awdf pra- 1 to 5 complete
21 Aug 2026
Medium
K
katariyajignesh266@gmail.com
Active Account
USER ID
6ab409c09a9ac4b542f53f85
EMAIL
katariyajignesh266@gmail.com
MEMBER SINCE
23 September 2026
PASSWORD
Secured & encrypted
Close




```
PS C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Import
All-AWDF- Practical
└── https://github.com/sponsors/feross
    └── safe-buffer@5.2.1

PS C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Import
```

task-manager-p6 > backend > models > User.js > ...

```
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  email: {
    type: String,
    required: [true, 'Please provide an email address'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [
      /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,})+$/,
      'Please provide a valid email address'
    ]
  },
  password: {
    type: String,
    required: [true, 'Please provide a password'],
    minlength: [6, 'Password must be at least 6 characters long']
  },
  createdAt: {
    type: Date,
    default: Date.now
  }
});

module.exports = mongoose.model('User', userSchema);

```

task-manager-p6 > backend > middleware > auth.js > auth > error

```
const jwt = require('jsonwebtoken');
const auth = (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. No token provided or malformed authorization header (expected "Bearer <token>").'
    });
  }

  const token = authHeader.split(' ')[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. Token is missing.'
    });
  }

  try {
    const secret = process.env.JWT_SECRET;
    if (!secret) {
      console.error('FATAL: JWT_SECRET environment variable is not defined.');
      return res.status(500).json({
        success: false,
        error: 'Server configuration error: JWT_SECRET missing.'
      });
    }

    const decoded = jwt.verify(token, secret);
    req.user = decoded;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Token has expired. Please log in again.'
      });
    }
    return res.status(401).json({
      success: false,
      error: 'Invalid token. Authorization denied.'
    });
  }
};

module.exports = auth;
```

task-manager-p6 > backend > middleware > validateInput.js > ...

```
const EMAIL_REGEX = /^\w+([.-]?\w+)*@\w+([.-]?\w+)*(\.\w{2,})+$/;


const validateRegister = (req, res, next) => {
  const { email, password } = req.body || {};
  const errors = [];

  if (!email || typeof email !== 'string' || !email.trim()) {
    errors.push('Email is required');
  } else if (!EMAIL_REGEX.test(email.trim())) {
    errors.push('Please provide a valid email address');
  }

  if (!password || typeof password !== 'string') {
    errors.push('Password is required');
  } else if (password.length < 6) {
    errors.push('Password must be at least 6 characters long');
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      error: errors.join(', ')
    });
  }

  next();
};

const validateLogin = (req, res, next) => {
  const { email, password } = req.body || {};
  const errors = [];

  if (!email || typeof email !== 'string' || !email.trim()) {
    errors.push('Email is required');
  }
  if (!password || typeof password !== 'string') {
    errors.push('Password is required');
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      error: errors.join(', ')
    });
  }

  next();
};
```

```
const validateTaskInput = (req, res, next) => {
  const { title, priority } = req.body || {};
  const errors = [];

  if (req.method === 'POST') {
    if (!title || typeof title !== 'string' || !title.trim()) {
      errors.push('Title is required and cannot be empty');
    }
  }

  if (req.method === 'PUT' && title !== undefined) {
    if (typeof title !== 'string' || !title.trim()) {
      errors.push('Title cannot be an empty string');
    }
  }

  if (priority !== undefined) {
    const validPriorities = ['low', 'medium', 'high'];
    if (!validPriorities.includes(priority)) {
      errors.push(`Invalid priority "${priority}". Priority must be low, medium, or high.`);
    }
  }

  if (errors.length > 0) {
    return res.status(400).json({
      success: false,
      error: errors.join(', ')
    });
  }

  next();
};

module.exports = {
  validateRegister,
  validateLogin,
  validateTaskInput
};
```

Key Questions / Analysis	**Q1. Why must passwords be hashed before storage instead of saved as plain text, even in a lab/demo project?** **Ans:** Hashing protects passwords if the database is leaked. Plain-text passwords can be directly read and misused. **Q2. What does the authentication middleware actually verify, and what happens if the token is missing or expired?** **Ans:** It verifies that the authentication token is valid and not expired. If the token is missing or expired, the request is rejected, usually with **401 Unauthorized**. **Q3. Why should input validation happen on the server even if the frontend already validates the same fields?** **Ans:** Frontend validation can be bypassed. Server-side validation ensures that only valid and safe data reaches the database or backend logic.
Learning Outcome	Students will be able to implement a complete authentication flow using JWT, protect backend routes with middleware, and enforce server-side input validation.